

UMA: A Family of Universal Models for Atoms

Brandon M. Wood^{1,†,*}, Misko Dzamba^{1,*}, Xiang Fu^{1,*}, Meng Gao^{1,*}, Muhammed Shuaibi^{1,*}, Luis Barroso-Luque¹, Kareem Abdelmaqsoud², Vahe Gharakhanyan¹, John R. Kitchin², Daniel S. Levine¹, Kyle Michel¹, Anuroop Sriram¹, Taco Cohen¹, Abhishek Das¹, Ammar Rizvi¹, Sushree Jagriti Sahoo¹, Zachary W. Ulissi¹, C. Lawrence Zitnick^{1,†}

¹FAIR at Meta, ²Department of Chemical Engineering, Carnegie Mellon University

*Co-first Author, [†]Co-corresponding Author

The ability to quickly and accurately compute properties from atomic simulations is critical for advancing a large number of applications in chemistry and materials science including drug discovery, energy storage, and semiconductor manufacturing. To address this need, Meta FAIR presents a family of Universal Models for Atoms (UMA), designed to push the frontier of speed, accuracy, and generalization. UMA models are trained on half a billion unique 3D atomic structures (the largest training runs to date) by compiling data across multiple chemical domains, e.g. molecules, materials, and catalysts. We develop empirical scaling laws to help understand how to increase model capacity alongside dataset size to achieve the best accuracy. The UMA small and medium models utilize a novel architectural design we refer to as mixture of linear experts that enables increasing model capacity without sacrificing speed. For example, UMA-medium has 1.4B parameters but only ~50M active parameters per atomic structure. We evaluate UMA models on a diverse set of applications across multiple domains and find that, remarkably, a single model without any fine-tuning can perform similarly or better than specialized models. We are releasing the UMA code, weights, and associated data to accelerate computational workflows and enable the community to continue to build increasingly capable AI models.

Models: <https://huggingface.co/facebook/UMA>

Code: <https://github.com/facebookresearch/fairchem>

Correspondence: B.M.W. (bmwood@meta.com), C.L.Z. (zitnick@meta.com)



1 Introduction

Density Functional Theory (DFT) models the interaction of atoms from first principles through the estimation of their electronic structure. It serves as the foundation of modern computational chemistry and materials science, and has provided insights into many applications including drug discovery [70, 77], energy storage [28, 72, 73], and semiconductors [63, 76]. Despite DFT’s widespread adoption, its considerable computational expense limits its usage.

Machine learning models offer the potential to accurately approximate DFT while being dramatically faster ($O(n)$ vs. $O(n^3)$ for DFT, where n is the number of atoms); reducing computation time from hours to less than a second. Ideally, these models – referred to as Machine Learning Interatomic Potentials (MLIPs) – would generalize across the many tasks that utilize DFT. We refer to a “task” as a chemical domain (e.g. materials, catalysts, molecules) plus the specialized DFT settings required by domain. For each task, we can explore different applications of DFT, such as molecular dynamics, relaxations, vibrational analysis, and mechanical response. Training MLIPs that generalize across such tasks remains an open problem.

The scaling of datasets and model sizes has led to major breakthroughs in language and vision models, enabling their generalization across diverse data distributions and tasks [2, 25]. A similar scaling of atomic datasets is more challenging due to their computational cost, resulting in models typically being trained on smaller problem-specific datasets [17, 18, 57]. A potential solution is to pool data across tasks to allow for the creation of exceptionally large datasets for training multi-task models. Recently, several large domain-specific datasets including catalysts [11, 72], materials [5, 68], and molecules [45] have been released that, when combined, total

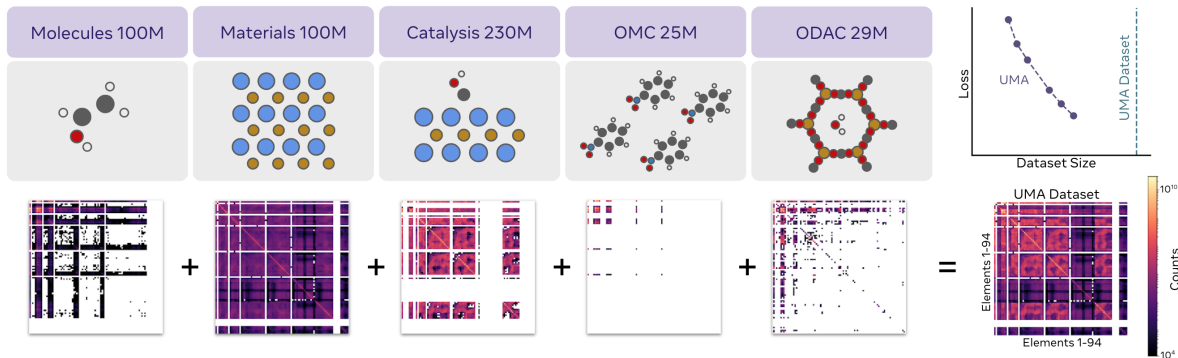


Figure 1 Visualization of the different datasets used for training. The 2D plots (bottom) illustrate the number of pairwise interactions contained in each dataset for every combination of elements. Note their combination covers nearly the entire chemical space with the exception of the radioactive elements. Model accuracies have improved with training dataset size (upper right), and this paper explores the limits of this scaling.

nearly half a billion atomic systems. This combined dataset defines a new paradigm in terms of the diversity of interactions and chemical environments (Figure 1), allowing for the learning of general-purpose models.

In this paper, we present a family of Universal Models for Atoms (UMA) designed to test the limits of accuracy, speed, and generalization for a single model across chemistry and materials science. The unprecedented amount of data ($\sim 500\text{M}$ atomic systems, Figure 1) poses new challenges in balancing accuracy and efficiency with multi-task training. To explore this space, we develop empirical scaling laws, relating compute, data, and model size, to determine the model size required to fit the UMA dataset and to define compute-optimal and inference-optimal training strategies. To accommodate the growing demand for model capacity while maintaining speed, we introduce a novel Mixture of Linear Experts (MoLE) architecture that efficiently scales the model size without increasing inference times for applications such as Molecular Dynamics (MD). For efficient model training, we propose a novel two-stage training schedule that efficiently pre-trains a model that is then refined in the second stage to conserve energy at higher precision.

We demonstrate that UMA, without fine-tuning to specific tasks, performs similarly or better in both accuracy and inference-speed/memory-efficiency than specialized models on a wide-range of material, molecular and catalysis benchmarks. Highlights include state-of-the-art results on the popular Matbench Discovery leaderboard [59], a 25% improvement in successful adsorption energy calculations for catalysis [43], and accuracy sufficient for practical applications (e.g. ligand-strain energy) in structure-based drug-design [45]. All code and data used for training UMA will be made publicly available. UMA model weights will be available with a commercially permissive license (with some geographic and acceptable use restrictions).

2 Approach

An MLIP acts as a surrogate for DFT, and so it requires the same inputs as DFT: atom positions, their atomic numbers, and optionally spin and charge information. As outputs, MLIPs estimate the energy of an atomic structure from which other properties may be computed through the use of derivatives, such as per-atom forces, stress, etc. For many applications, such as molecular dynamics or performing relaxations, an MLIP is used to calculate the atomic forces to run simulations that can require thousands or even millions of iterations. For this reason, MLIPs must be both accurate and computationally efficient.

2.1 UMA: Universal Model for Atoms

In this paper, we describe a family of models that can be categorized into three sizes: small (S), medium (M), and large (L). A summary of the models can be found in Table 1. All the models have different accuracy and speed attributes and as a result are best suited for distinct use cases. UMA-S aims to strike a balance between accuracy and efficiency and is suitable for computationally intensive applications such as long molecular dynamics simulations. The most general-purpose model of the family is UMA-M, which is more accurate

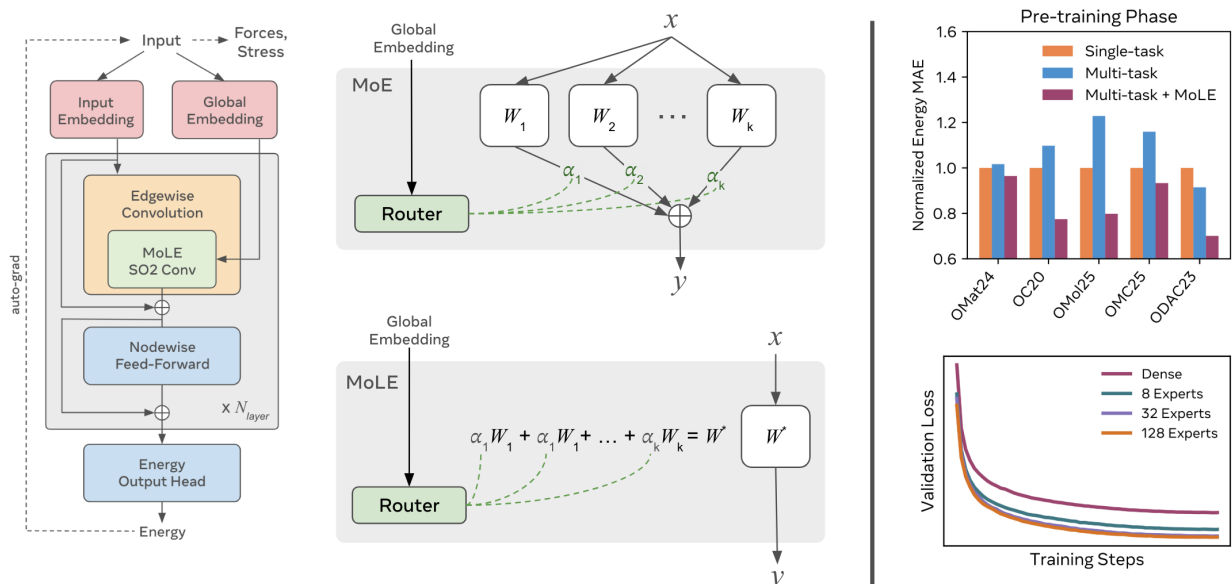


Figure 2 (left) Overview of UMA model architecture. The SO2 convolution is made up of a set of linear operations and each one of these operations is replaced with MoLE. (middle) Illustration of MoE and MoLE. The embedding used for routing, which estimates the expert weights α , only depends on global information making it possible to merge before the model forward pass (middle, bottom), which has substantial benefits for applications that require long roll outs such as molecular dynamics. (right) Bar plot of UMA-S trained with MoLE for multi-task and without MoLE for single and multi-task. Note the MoLE model outperforms non-MoLE models. (right, bottom) Loss plots when varying the number of experts from 1 to 128 for UMA-S.

Table 1 Summary of UMA models.

Model	Total Parameters	Active Parameters	Inferences per second for 1k Atoms	Max Atoms per 80GB GPU	Conservative
UMA-S	150M	6M	16	100k+	✓
UMA-S-1.2	290M	6M	24	100k+	✓
UMA-M	1.4B	50M	3	10k+	✓
UMA-L	700M	700M	1.6	1k+	✗

Inference speed and max atoms measured on Nvidia H100 with a periodic system that has ≈ 50 neighbors per atom within 6Å, see section D. Update March 2026 - UMA-S, UMA-S-1.1, UMA-M, UMA-L are benchmarked using the code from the original UMA release (fairchem>2.1.0). UMA-S-1.2 requires fairchem>2.16.0 and contains significant inference optimizations that are mostly compatible with older versions of UMA as well but we are not updating those previous numbers for record keeping. UMA-S-1.1 has identical architecture as UMA-S and similar inference speeds as UMA-S-1.2 if run with fairchem>2.16.0.

and can be used as a DFT surrogate for relaxations or vibrational analysis as well as for molecular dynamics. UMA-L is a highly accurate DFT surrogate and is intended as a proof-of-principle to help understand scaling behavior and to demonstrate the limits of what is currently feasible.

We begin by providing a basic introduction to the UMA architecture. Next, we highlight our use of Mixture of Linear Experts (MoLE), an efficient way to scale capacity and add flexibility for training models across different DFT datasets and tasks. Lastly, we present an overview of the training procedure used for UMA models.

2.2 Architecture

The UMA architecture is based on eSEN [23], an equivariant graph neural network, with a number of important modifications to enable the model to efficiently scale and handle additional inputs, such as total charge and spin, and the DFT settings desired for emulation. The standard eSEN architecture takes 3D atomic positions and atomic numbers as inputs and returns the total energy, per-atom forces, and, optionally, stress. The network operates by updating a spherical harmonic node embedding for each atom through a series of message passing layers. The embeddings are initialized based on the atom’s atomic number. Messages are

passed between neighboring atoms that are less than a predefined distance (6 Å) from each other. Each message passing layer consists of an edge-wise block followed by a node-wise feed-forward block with residual connections. Normalization is performed after each layer. The central component of the edgewise block is the eSCN convolution [54]. After message passing, there is a single task-independent node-wise feed-forward block to predict the outputs.

2.2.1 Charge, Spin, and DFT Task Inputs

In addition to the inputs handled by eSEN, we expand the model to incorporate information about the system’s total charge, total spin multiplicity, and DFT task. The charge and spin are indicated using an integer value for each and allow the MLIP to model structures which may be electrically charged (have extra or missing electrons) or have unpaired electrons. The DFT task indicates which of the five training dataset DFT settings the model should attempt to replicate. A single dataset is specified and a random vector corresponding to it is fed into the network. This is necessary since different datasets use different plane-wave or localized orbital DFT calculators (VASP [39, 40] and ORCA [51]) and levels of theory.

We introduce a new embedding to UMA models that enables the inclusion of charge, spin, and DFT task. Each of these inputs generates an embedding the same dimension as the spherical channels used, which is concatenated and then fed through a 1-layer feed-forward network. The result is added to the node embeddings at each layer for the spherical harmonics coefficients of degree 0 ($L = 0$). This embedding is also used to compute the global embedding used for MoLE routing, which is described below.

2.3 Mixture of Linear Experts

A common approach to learning general-purpose models is to increase the amount and diversity of their training data. As datasets increase in size, scaling laws suggest model sizes must also be scaled to optimally reduce losses [2, 25]. However, this presents a tradeoff, which may lead to accurate models that are too computationally expensive to use in practical applications. One method to improve these trade-offs is to use Mixture of Experts [32, 33, 49] (MoEs) to increase the number of model parameters while minimizing the additional computational cost. In an MoE model, the outputs of a block are calculated by a set of experts, each with their own individual set of weights. Typically, a sparse weighted combination of their outputs is combined and passed to the next block. This approach has been shown to work well for LLMs to improve both their efficiency and their ability to generalize [15, 21, 34, 60].

The application of MoEs to MLIPs requires additional considerations. Contrary to language modeling, estimating the potential energy surface is a regression task whose outputs should vary smoothly to ensure energy conservation [23]. In addition, the estimated forces should be equivariant to rotation. This implies that the use of experts should not introduce discontinuities on the energy surface, and care must be taken to ensure equivariance is maintained. Another practical consideration for MLIPs is that the task and set of atoms is commonly held constant during long simulations. Ideally, an MoE approach for MLIPs would take advantage of this to improve efficiency in sequential inference applications.

We propose using a Mixture of Linear Experts (MoLE) for MLIPs. An MoLE combines a set of linear experts [32, 48]:

$$y = \sum_k \alpha_k (W_k x) \quad (1)$$

where each expert k has a set of weights W_k and contribution $\alpha_k \in [0, 1]$. While this was one of the original approaches proposed for MoEs [32], MoEs that use sparse sets of non-linear experts which compete for attention [33] are much more commonly used in modern networks [15, 21, 60, 64]. However, MoLEs offer distinct advantages for MLIPs. First, the MLIP can learn functions that share information and vary smoothly between tasks by encouraging the dense use of all experts without enforcing sparseness. Second, since MoLE is a mixture of linear experts, they maintain rotational equivariance when used within the eSCN convolution [54]. Finally, if the expert weights are only dependent on time-invariant global information such as element composition, the network weights may be precomputed before running simulations [48, 75]. The precomputed weights W^* are calculated by moving the summation in Equation 1, which results in MoE

inference times that are similar to non-MoE models:

$$y = W^*x \quad \text{where} \quad W^* = \left(\sum_k \alpha_k W_k \right) \quad (2)$$

The contribution α_k of each expert is calculated as a function of system-level features, including element composition, charge, spin, and task information. Embeddings are calculated for each of these properties, concatenated, and passed through a 3-layer MLP followed by a softmax to estimate α_k , $\sum_k \alpha_k = 1$. We specifically exclude information when calculating α 's that may vary during the course of relaxations or MD simulations, such as relative atom positions or other neighborhood information.

2.4 Training Procedure

Training models on large datasets across numerous tasks is challenging due to the computational resources needed. This is especially true for conservative models, which require an additional backward pass to calculate forces or stress. To improve training efficiency, we implemented a two-stage approach described in detail in the supplementary material (Appendix A). As proposed by [7, 23], we train a model that directly predicts forces in the first stage. In the second stage, we remove the force head and fine-tune the model to predict conserving forces and stresses using auto-grad. This approach takes advantage of the faster training enabled by direct models, while providing energy conservation and smooth potential energy landscapes required by many applications.

Several other novel improvements were made to each stage’s training to further improve efficiency. Half-precision is critical to training efficiently on modern GPUs. Similar to other domains such as LLM training, we found that BF16 is significantly more stable compared to FP16 with automatic mixed precision. However, unlike LLMs, MLIPs are significantly more sensitive to numerical precision; we observed that BF16 alone will degrade accuracy by as much as 20 – 50% depending on the task and the property being computed. We found that pre-training with BF16 and switching to FP32 for fine-tuning recovers the loss in accuracy. Finally, training the models with a large number of MoLE experts is challenging due to memory constraints. We make further optimization to help bound and amortize memory usage (Appendix A). In addition, models were trained with a combination of graph parallelism [67] and fully-sharded data parallelism for large MoLE layers and activation checkpointing (Appendix A). When combined, this allows us to reliably scale up model training up to 10B total parameters scale.

The energies for each dataset can vary significantly due to the use of different DFT settings, which makes multi-task training difficult. To account for this, an energy referencing scheme was employed as described in Appendix A.6. This approach allowed for the use of a single energy head across all tasks without the need for task-specific heads.

3 Datasets

To train a model capable of generalizing across DFT tasks, an ideal training dataset would include materials, molecules, and their interactions. The Open Molecules 2025 (OMol25) [45] and Open Materials 2024 (OMat24) [5] datasets cover both molecules and materials respectively. The Open Catalyst 2020 (OC20) [11] and OpenDAC 2025 (ODAC25) [20] datasets are useful for modeling the interactions of molecules and materials. Finally, the Open Molecular Crystals 2025 (OMC25) [19] dataset specializes in modeling the interaction between molecules in periodic structures. In combination, these datasets contain close to 500 million training examples with over 30 billion atoms. As visualized in Figure 1, the combination of these datasets contains nearly all pairwise interactions between atoms of different elements. We focused on large datasets, because of the challenges of mixing different DFT tasks and since the inclusion of much smaller datasets was unlikely to significantly impact the model weights.

While all DFT calculations yield energy and force labels by determining ground-state electronic structures, different chemical systems require domain-specific DFT settings. For instance, the OMat24 dataset uses the PBE functional and is generated with the plane-wave code VASP, whereas OMol25 employs the ω B97M-V functional with the localized orbital code ORCA. Designing a single model that performs well across such